Digital Professor: Interaction Methods

Research Reference Notes and Prototype Concept

Digital Professor Research Project

Research snapshot: September 5, 2026

1 Purpose and recommendation

The broader project aims toward a course-agnostic digital professor or teaching assistant. These notes examine its *interaction layer*: how students ask questions, share their work, and receive feedback. Statistics and engineering provide demanding initial examples because explanations frequently combine prose, equations, plots, and intermediate solution steps. Multimodal generative AI can connect these representations, but interface capability alone does not establish teaching competence or AGI.

Recommended initial experience: a browser-based shared worksheet with text chat, selected PDF pages, photo uploads, an editable equation field, and optional push-to-talk with spoken playback. Add a simple stylus canvas that submits a selected region when the student requests feedback. Keep live conversational audio and continuous video as subsequent experiments.

Scope and evidence. This is a focused review of official API documentation and maintainer documentation, accessed on the date above. Citations establish advertised functionality; feasibility ratings, sequencing, and evaluation targets are project judgments, not measured benchmarks. Assume a small student team, ordinary laptops/tablets, internet access, modest paid API use, and no model training or dedicated GPU requirement. Exact prices, account availability, model identifiers, and preview status should be checked when implementation begins. The proposed concept is a reversible starting point, not a final architecture.

2 Comparison at a glance

Here, *high feasibility* means a bounded version can reuse standard interfaces or hosted services; *medium* means recognition quality or device/session integration needs focused experimentation; *lower* means substantial integration relative to the proposed first demonstration. Effort is relative engineering effort, not a time estimate. Cloud services introduce usage charges and data transfer; locally runnable libraries introduce maintenance and hardware costs.

Dimension	Candidate technologies	Value and principal constraint	Initial feasibility
Text	Responses API; browser chat; KaTeX [1, 18]	Precise, searchable dialogue; generated explanations still need grounding and checking.	High; low effort. Core.

Dimension	Candidate technologies	Value and principal constraint	Initial feasibility
Voice conversation	OpenAI Realtime; Gemini Live; LiveKit Agents [2, 3, 4]	Natural follow-ups and interruptions; turn detection, reconnects, and shared visual context add work.	Medium; medium–high effort. Later.
Speech recognition / response	MediaRecorder; hosted transcription and TTS; faster-whisper; Web Speech API [5, 6, 7, 8, 9]	Spoken questions and read-aloud hints; symbols, accents, noise, and playback require testing.	High for push-to-talk; low–medium effort. Optional early.
Long-form audio	File transcription; timestamped segments; persistent session notes [6, 8]	Review an explanation or resume tutoring; chunking and summaries can lose details.	High for bounded recordings; medium effort. After core.
Documents	PDF.js; direct file input; Docling; hosted file search [10, 11, 13, 12]	Discuss notes and student work; reading order, scans, and citation accuracy vary.	High for selected pages; medium effort. Core.
Handwriting / stylus	Pointer Events + canvas; MyScript iinkTS; Mathpix [14, 15, 16]	Preserve derivations and sketches; ink capture is easier than reliable interpretation.	High for snapshots, medium for recognition. Early extension.
Mathematical expressions	MathLive; KaTeX; Mathpix; SymPy [17, 18, 16, 19]	Structured input, display, and bounded checking; notation and assumptions remain ambiguous.	High for entry/display; medium for checking. Core.
Image / camera	File upload; getUserMedia; vision model; Mathpix [20, 21, 16]	Share paper, plots, or diagrams; tiny labels and spatial reasoning can fail.	High for still images; low–medium effort. Core.
Tablet / iPad	Responsive web UI; Pointer Events; native PencilKit [14, 22]	Read, write, and discuss together; touch, keyboard, palm handling, and browser behavior need device testing.	High for web access; medium for polished pen UX. Core web.
Video	Gemini video understanding; Gemini Live; MediaRecorder [23, 3, 5]	Explain an evolving procedure; sampling, bandwidth, and temporal alignment complicate feedback.	Medium for clips; lower for continuous tutoring. Later.

3 Detailed technology notes

3.1 Text-based interaction

An ordinary chat interface can send messages to a hosted language model through an API such as OpenAI Responses; the official guide documents text generation and SDK examples.[1] Proposed interaction controls are *Explain*, *Give a hint*, *Check this step*, and *Quiz me*. These represent instructional intentions rather than separate models. Text is the most useful baseline because students can revise notation, inspect prior feedback, and use the interface without a microphone.

Store the selected course, source references, student attempt, and recent dialogue together. A course

package should supply the syllabus, terminology, learning objectives, and allowed solution conventions without changing the interface. Initially choose one hosted text-and-image model after a small task comparison; avoid training a course-specific model. Stream responses if helpful, but show final equations only after parsing/rendering succeeds. Render math with KaTeX and retain a readable text equivalent. KaTeX renders expressions; it does not verify them.[18]

Feasibility judgment: high. The main uncertainty is pedagogical correctness rather than chat construction. Ask for an attempted step before giving a full solution, ground course-specific statements in supplied materials, and expose uncertainty when the materials do not answer the question.

3.2 Voice: real-time and long-form conversation

Two approaches deserve separate evaluation. A *cascade* transcribes speech, sends text to the tutoring model, and synthesizes its reply. A *native speech-to-speech session* handles conversational audio directly. OpenAI Realtime documents low-latency multimodal interaction and WebRTC/WebSocket connections. Gemini Live accepts streams of audio, images, and text and produces spoken responses; its overview currently labels the API Preview.[2, 3] These are service capabilities, not guarantees of latency or mathematical accuracy in our setting.

For an initial cascade, use an explicit recording button and show the recognized question before submission. This gives students a correction point for "sigma squared" or "x sub i." It also reuses the text tutoring path. Real-time voice is attractive for office-hours dialogue, but needs interruption handling, end-of-turn detection, cancellation of obsolete answers, and coordination with the current worksheet. A student pausing to calculate should not automatically trigger a new explanation.

LiveKit Agents provides a framework for voice-agent sessions and integrations.[4] It becomes useful if session orchestration dominates the work; a single-provider push-to-talk demo may not need that extra layer. Use server-issued short-lived credentials when a browser connects directly to a provider; permanent API credentials belong on the backend.[2, 3]

Long-form has two meanings. For a recorded student explanation, transcribe a bounded upload, retain segment references, and let the student ask about a particular passage. For a long tutoring conversation, preserve a compact progress summary plus the actual equations and cited artifacts; a summary alone may erase a critical assumption. Split lengthy recordings according to the chosen endpoint's limits and test transitions between chunks. File transcription and local faster-whisper support the transcription component.[6, 8] Long duration does not imply persistent, reliable model memory.

Feasibility judgment: high for short recorded turns, medium for long sessions, and medium with greater integration effort for fluid real-time conversation. Measure response delay on the actual device and network before promising conversational performance.

3.3 Speech recognition and spoken responses

Browser MediaRecorder captures media for upload; it is not a speech recognizer.[5] Hosted candidates include `gpt-4o-transcribe` or `gpt-4o-mini-transcribe` for recognition, and `gpt-4o-mini-tts` for spoken replies. Their guides document transcription and speech generation separately.[6, 7] This separation allows the project to inspect recognized text and choose what the tutor speaks.

For a locally runnable alternative, faster-whisper implements Whisper inference using CTranslate2 and supports CPU/GPU configurations and quantization.[8] This offers deployment control but requires testing throughput on available hardware. Do not assume local inference is faster or cheaper at the

project's scale.

The Web Speech API exposes recognition and synthesis, including evolving on-device recognition features.[9] Browser and language support must be checked on target devices; it is a useful convenience experiment rather than the sole required input path. Always retain typing. For output, speak a short explanation and keep the full derivation visible. Translate notation into spoken language instead of reading raw LaTeX, provide stop/replay controls, and identify the voice as AI-generated.[7]

Feasibility judgment: high for optional audio. Test mathematical terms, negation, accented speech, background noise, and headphone-free playback. A low overall word error rate can still conceal a consequential error such as losing "not" or a minus sign.

3.4 Reading documents and student-provided content

PDF.js supplies browser PDF viewing.[10] A direct model file-input route can use PDF text and page imagery, while hosted file search retrieves relevant information from uploaded files using semantic and keyword search.[11, 12] These serve different interaction needs: selected pages provide immediate visual context; retrieval helps locate material across a larger course collection.

Docling is a locally runnable document-conversion option with layout, table, and OCR capabilities.[13] It offers more control over ingestion but adds a processing pipeline. Mathpix is a specialized option for printed or handwritten STEM content in images, PDFs, and strokes.[16] Neither successful conversion nor retrieval establishes that the tutor understood the material correctly.

Prototype route: support pasted text and a small PDF, display the selected page, and attach that page or region to the question. Preserve document ID and page number so the answer can link back to evidence. Add retrieval when the demonstration actually needs a course corpus. Retain page images for diagrams and equations because extracted text alone can lose their relationships. For numerical tables or later CSV support, preserve cells and units and use a computation tool for calculations instead of asking vision to estimate values.

Feasibility judgment: high for bounded documents, medium for heterogeneous course ingestion. Test scans, multi-column pages, equations, and tables separately. Keep course collections isolated, distinguish a student's attempt from authoritative notes, and treat uploaded instructions as source content rather than instructions controlling the application.

3.5 Handwriting and stylus/pencil input

Separate *capture*, *recognition*, and *interpretation*. Pointer Events unify mouse, touch, and pen input and expose properties such as pressure and tilt where supported.[14] A canvas can preserve strokes and export an image without understanding anything written on it.

MyScript iinkTS provides browser ink capture/rendering and server-backed handwriting recognition, including text and mathematical content. Its documentation notes that recognition depends on server communication and network latency.[15] Mathpix accepts both image and stroke inputs for STEM recognition.[16] Evaluate these commercial services if a general vision model cannot reliably recover the notation; account for credentials, usage costs, and network dependence.

Prototype route: save the original ink, let the student select a region, and send a snapshot on *Check this step*. Display "I read this as . . . " with an editable expression before checking an ambiguous result. Preserve stroke order for later experiments with step-by-step feedback. Continuous recognition after every pen movement introduces distracting corrections and avoidable requests.

Feasibility judgment: high for capture and snapshots; medium for dependable recognition. Build a small handwriting set covering fractions, subscripts, matrices, Greek letters, crossed-out work, and sketches. The ability to recognize a symbol does not imply the ability to assess the student's reasoning.

3.6 Mathematical equations and expressions

MathLive provides a web math field with a virtual keyboard and LaTeX-oriented input/output; KaTeX renders mathematical notation.[17, 18] Together they offer a typed correction path for voice, OCR, and handwriting. OCR services recover notation; a computer algebra system such as SymPy can support bounded symbolic checks. SymPy's parsing documentation warns that its string parser uses `eval` and should not receive unsanitized input.[19]

A proposed mathematical pipeline is: preserve the original input, display the recognized expression, confirm ambiguities, convert supported notation to a constrained representation, and run a narrowly scoped check. Execute computation in an isolated worker with resource limits. Do not send arbitrary student strings directly to a general evaluator.

For example, matching two algebraic forms does not establish that a confidence interval uses the right distribution, degrees of freedom, or assumptions. Likewise, an engineering result needs units and boundary conditions. Begin with a small declared set of checks and permit "unable to verify." Treat LaTeX as presentation, not a complete specification of mathematical meaning.

Feasibility judgment: high for entry and display; medium for reliable checking. Structured input should be part of the first prototype even if symbolic verification is deferred.

3.7 Image and camera input

An upload field covers photographs of handwritten work, screenshots, plots, and engineering diagrams. For an in-app camera, `getUserMedia` requires user permission and a secure context.[20] A vision-capable model can interpret images, but official documentation notes limitations including small text, rotation, graphs, and precise spatial reasoning.[21]

Start with still-image upload, preview, crop, and retake controls. Ask which region the student wants explained and retain that selection in the conversation. Use OCR when exact symbols matter; use multimodal interpretation when the relationship between prose, equations, and a diagram matters. If a plot value is critical, request the underlying data or a clearer crop.

Feasibility judgment: high. It provides a direct path from existing paper work into tutoring without requiring a custom handwriting editor. Evaluate the photographed content, not the student's face or presumed emotional state.

3.8 Tablet and iPad interaction

A tablet is a device context combining touch, pen, keyboard, camera, and microphone. A responsive browser application can reuse the same course session and interaction components across laptops and iPads. Pointer Events provide the web input foundation.[14] Test scrolling versus drawing, orientation changes, keyboard obstruction, audio playback, and recovery when the browser backgrounds the page. Pressure and palm behavior should be tested on actual devices rather than inferred from API presence.

For a native Apple implementation, PencilKit offers drawing views, tools, and stroke data. Its current

documentation also lists `PKStrokeRecognizer` handwriting recognition as Beta.[22] That is an emerging option for on-device text recognition, not evidence of general mathematical reasoning. Verify supported OS versions, languages, and deployment status before adopting it.

Feasibility judgment: high for a responsive web prototype; medium for a polished native pen experience. Start with one device running both worksheet and conversation. A paired laptop-plus-tablet experience requires additional session pairing and synchronization and can follow if user testing justifies it.

3.9 Video-based interaction

Distinguish prerecorded clips, live visual input, and generated video output. Gemini documents video understanding and configurable frame sampling.[23] An uploaded clip could support "What went wrong at this point in my experiment?" or discussion of a recorded solution. Keep timestamps and let students select the relevant interval. Sparse sampling may miss a short action or a quickly erased equation; a fluent summary does not demonstrate frame-by-frame understanding.

Gemini Live supports real-time voice and visual interaction but is currently marked Preview.[3] Continuous tutoring adds bandwidth, interruptions, temporal alignment, and the need to associate spoken references such as "this line" with the right visual state. OpenAI Realtime's image input should not be assumed to mean an arbitrary video-file endpoint.[2]

Feasibility judgment: medium for bounded clips; lower priority for continuous video. An initial substitute is a paused frame plus a typed or spoken question. For output, prioritize equations, plots, and replayable explanations; an animated professor or generated lecture video can be explored later if embodiment is an explicit research question. This review does not recommend an avatar dependency.

4 Lightweight concept: a shared office-hours worksheet

The student opens a course workspace, selects a problem or page, and works beside a tutor conversation. All input modes refer to the same selected artifact. The sketch below is an interaction proposal; it does not prescribe a frontend framework, cloud provider, or storage system.

Example student journey.

1. Select a course and attach its notes. Open a homework problem and upload or write an attempted solution.
2. Select a line and ask, by text or recorded voice, "Why is this step wrong?" Review the transcript or recovered equation if necessary.
3. The tutor receives the question, selected region, confirmed notation, and relevant course material. It asks about assumptions before judging the step.
4. For a small independent normal sample with unknown population standard deviation, discuss the appropriate Student's t interval and its assumptions. The tutor links to the supplied course explanation rather than inventing a page citation.
5. The student revises the expression to $\bar{x} \pm t_{1-\alpha/2,n-1} s/\sqrt{n}$, explains the choice, and requests feedback. The tutor gives a follow-up question and saves a short progress note.

The same interaction can discuss a free-body diagram or a written argument by changing course materials and evaluation criteria. This is the operational meaning of course-agnostic: reusable interaction and context handling, with domain-dependent correctness checks.

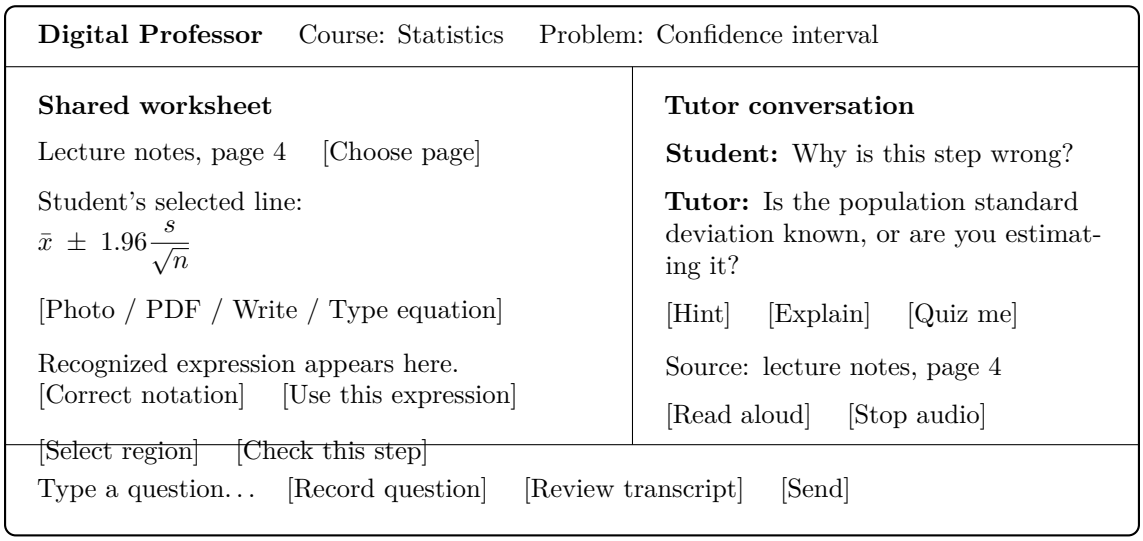
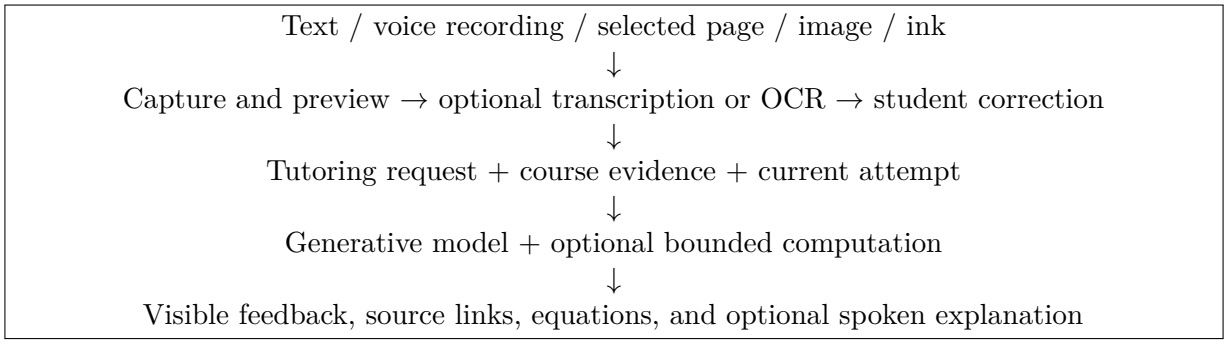


Figure 1: Proposed tablet landscape or desktop view. On narrow screens, switch between worksheet and conversation while preserving the selection. Example dialogue is illustrative, not output from an evaluated system.

Minimal conceptual flow:



Keep these boundaries replaceable. A useful request record contains a session ID, course ID, question, attachment/page/region references, original input, corrected notation, and requested feedback mode. Provider-specific adapters can change without rewriting the student's workflow. Begin with one backend service; a distributed or multi-agent architecture is not needed to test this interaction concept.

5 Prototype priorities and feasibility tests

1. **First demonstrable slice:** text dialogue, one selected PDF page or photo, rendered and editable equations, source-linked feedback, and laptop/iPad web access. Demonstrate a full attempt–hint–revision loop in statistics and an engineering topic.
2. **Small extensions:** push-to-talk, optional TTS, and an explicit-submit stylus canvas. Each should fall back to the existing text/image path if recognition fails.
3. **Subsequent experiments:** corpus retrieval, long recordings, improved stroke recognition, and bounded symbolic/numerical checks. Add only the services needed by observed failures.
4. **Later research:** real-time interruptible voice, live camera/video tutoring, native PencilKit recognition, and synchronized multiple devices. Compare their learning and usability benefit against their added complexity.

Evaluation proposal, not reported results. Assemble 30–50 representative prompts spanning statistics and engineering, with typed references and matched audio, photo, or ink variants. Include unclear inputs and questions not answered by the supplied material. Use consenting pilot participants and preserve only the data needed for the study.

- **Recognition:** measure transcript word error rate, but separately score critical words, signs, subscripts, and full-expression correctness. Record how often users must correct input and how long correction takes.
- **Grounding and tutoring:** have a knowledgeable reviewer assess correctness, assumption checking, source support, hint usefulness, and whether feedback addresses the student's actual step. Score perception errors separately from reasoning errors.
- **Responsiveness and reliability:** report median and 95th-percentile time from submission to useful feedback, time to first audio, upload failures, and recovery after network interruption. An initial target might be useful text feedback within five seconds for a short request; this is an engineering target to validate, not a provider guarantee.
- **Cost:** log actual token, audio-duration, OCR-page, storage, and hosting usage per completed task. Compare total cost per successful tutoring interaction, including retries and corrections, rather than isolated model-call prices.
- **Learning and usability:** compare text-only with added modalities on matched tasks, counterbalance order, and collect completion time, correction burden, preference, and a short transfer question. Report uncertainty and participant-level variation; a small convenience pilot supports feasibility findings, not broad learning-effect claims.

An integration is worth keeping when it reduces student effort or improves demonstrated feedback quality. Log model/version and input conditions so results remain interpretable as services change. Make recording explicit, retain text alternatives, and define deletion and course-access behavior before using real student work. Full autonomous grading, institutional integration, and replacing every professor responsibility remain outside this initial interaction demonstration.

References

[1] OpenAI. *Text generation*. https://developers.openai.com/api/docs/guides/text.

[2] OpenAI. *Realtime and audio*. https://developers.openai.com/api/docs/guides/realtime.

[3] Google. *Gemini Live API overview*. https://ai.google.dev/gemini-api/docs/live-api.

[4] LiveKit. *Agents documentation: Introduction*. https://docs.livekit.io/agents/.

[5] Mozilla contributors. *MediaRecorder*. https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder.

[6] OpenAI. *File transcription*. https://developers.openai.com/api/docs/guides/speech-to-text.

[7] OpenAI. *Text to speech*. https://developers.openai.com/api/docs/guides/text-to-speech.

[8] SYSTRAN and contributors. *faster-whisper*. https://github.com/SYSTRAN/faster-whisper.

[9] Mozilla contributors. *Web Speech API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API.

[10] Mozilla. *PDF.js*. https://mozilla.github.io/pdf.js/.

[11] OpenAI. *File inputs*. https://developers.openai.com/api/docs/guides/file-inputs.

[12] OpenAI. *File search*. https://developers.openai.com/api/docs/guides/tools-file-search.

[13] Docling contributors. *Docling: document processing for generative AI*. https://github.com/docling-project/docling.

[14] Mozilla contributors. *Pointer events*. https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events.

[15] MyScript. *iinkTS: Get started*, Interactive Ink 4.1 documentation. https://developer.myscript.com/doc/interactive-ink/4.1/web/iinkts/get-started/.

[16] Mathpix. *Mathpix OCR API: Introduction*. https://docs.mathpix.com/.

[17] MathLive. *Mathfield: Introduction*. https://mathlive.io/mathfield/.

[18] KaTeX. *API*. https://katex.org/docs/api.

[19] SymPy contributors. *Parsing*. https://docs.sympy.org/latest/modules/parsing.html.

[20] Mozilla contributors. *MediaDevices: getUserMedia() method*. https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia.

[21] OpenAI. *Images and vision*. https://developers.openai.com/api/docs/guides/images-vision.

[22] Apple. *PencilKit*. https://developer.apple.com/documentation/pencilkit.

[23] Google. *Video understanding*. https://ai.google.dev/gemini-api/docs/video-understanding.